Compile, watch, serve

Compile

Use `calepin compile` when you want to execute code chunks and render a document. The command line shape is intentionally the same as `typst compile`: same output-first/format-driven arguments, with `--` pass-through for Typst flags, plus Calepin preprocessing.

```
calepin compile paper.typ --format pdf
calepin compile paper.typ --format html
calepin compile paper.typ {p}.svg --format svg

# explicit output path
calepin compile paper.typ path/to/paper.pdf --format pdf
```

Extract scripts

Use `--format script` to extract executable source code without running it or rendering the Typst document. Script extraction accepts one `.typ` input at a time; website directories and `calepin watch` are not supported.

```
# write paper.R, paper.py, paper.jl, or paper.sh as needed
calepin compile paper.typ --format script

# choose the output template
calepin compile paper.typ exported/paper.{ext} --format script
calepin compile paper.typ exported/{engine}.{ext} --format script
```

Calepin writes a separate file for each engine and preserves chunk order within that engine. It includes chunks with `eval: false`, but excludes diagram chunks. Recognized languages use conventional extensions,
including `.R`, `.py`, `.jl`, `.sh`, `.rs`, `.yaml`, `.js`, `.ts`, `.c`, `.cpp`, `.java`, `.go`, `.rb`, `.sql`, `.lua`, `.toml`, `.json`, `.html`, and `.css`. Unrecognized Jupyter kernels use `.txt`; use `{engine}` when more than one kernel would otherwise produce the same path.

Chunk-label separators use the recognized language's comment syntax: `#` for Python and YAML, `//` for Rust and JavaScript, `--` for SQL, and block comments for HTML and CSS, for example. Languages such as JSON that do not support comments, and unknown languages whose syntax Calepin cannot safely infer, are joined with blank lines and no label separator. For an unknown engine with an explicit output path, Calepin can infer known comment syntax from the file extension; a recognized engine name always takes precedence.

`{ext}` expands to the extension without its leading dot, while `{engine}` expands to a filename-safe engine name. When an explicit output has no placeholder, it is accepted only when the document contains one engine. An output template that maps multiple engines to the same path is rejected.

Choose chunks and output files

Use the `script` chunk option to exclude code from extraction or route chunks to named files. Chunks that name the same file are appended in document order.

````
```python
#| script: scripts/prepare.py
print("first part")
```

```python
#| script: false
print("shown in the notebook, but not exported")
```
````

````
```python
#| script: scripts/analyze.py
print("a different Python script")
```
````

The option accepts `true`, `false`, or a relative path. Missing options and `script: true` use the default engine-specific output described above. `script: false` excludes a chunk. A path writes to that exact destination, relative to the input `.typ` file's directory; absolute paths and paths that escape that directory are rejected.

To export only explicitly selected chunks, disable script extraction in the document setup and opt chunks back in individually:

````
#calepin.setup(script: false)

```yaml
#| script: docker-compose.yml
services:
```

```yaml
#| script: docker-compose.yml
  api:
    image: example/api
```
````

An output template passed on the command line applies only to chunks using the default engine-specific destination. Explicit `script: path` destinations are kept as written. Calepin reports an error if explicit and generated destinations resolve to the same file.

Compile a website by pointing `calepin compile` at a source directory that contains `calepin.toml`:

```
calepin compile docs docs
calepin compile my_site public
```

Arguments after `--` are forwarded to Typst, so project-specific Typst flags can stay in the same command.

```
# open PDF in system viewer
calepin compile paper.typ -- --open

# set path to font directory
calepin compile paper.typ -- --font-path fonts
```

Progress output

Calepin shows animated progress when the terminal supports it, using the same terminal detection as the underlying progress renderer. When output is redirected, the terminal is not interactive, or the terminal reports limited capabilities, Calepin falls back to plain status lines.

Some terminal panes and command runners report themselves as interactive but render spinner redraws as separate lines. Set `CALEPIN_PROGRESS` to `plain` to disable animated progress while keeping status messages:

```
CALEPIN_PROGRESS=plain calepin compile paper.typ
```

In PowerShell:

```
$env:CALEPIN_PROGRESS = "plain"
calepin compile paper.typ
```

Watch

Use `calepin watch` while editing. It watches your source for changes, re-runs preprocessing, and delegates recompilation and previewing to `typst watch`. The command form is the same as `typst watch`: same positional arguments and pass-through flags, with Calepin running its preprocessing step first.

```
calepin watch paper.typ
calepin watch paper.typ --format html
calepin watch paper.typ {p}.svg --format svg
```

The default output format is PDF. Choose another format with `--format`, or let the output extension select the format.

Arguments after `--` are passed through to `typst watch`. Typst's `--open` flag opens the rendered output in the operating system's default viewer, and Typst's `--port` flag chooses the HTML preview port.

```
calepin watch example.typ -- --open
calepin watch paper.typ paper.html --format html -- --port 3001 --open
```

Watch a website directory by passing the source and output directories:

```
calepin watch docs docs
calepin watch my_site public
```

Add `--serve` to run the local server while watching a website. It uses the same `--host`, `--port`, and `--open` options as `calepin serve`.

```
calepin watch docs docs --serve --open
calepin watch my_site public --serve --host 127.0.0.1 --port 8001
```

Eval-only watch

When Tinymist or another editor already provides Typst preview, use `--eval-only` to refresh Calepin's computational artifacts without starting a second `typst watch` process or writing a rendered output file:

```
calepin watch paper.typ --eval-only
```

Calepin performs an initial preprocessing pass, publishes the notebook for external preview, and then monitors changes. It runs a lightweight Typst query to detect chunk definitions, but Python, R, Jupyter, shell, and diagram engines run only when the computational fingerprint changes. Prose-only edits use the existing results cache; display-only chunk options can refresh stored rendering metadata without re-executing code.

This single-file mode does not accept an output path, `--format`, serving options, or Typst pass-through arguments. Tinymist remains responsible for rendering and live preview.

PDF viewer auto-refresh

Some PDF viewers do not automatically refresh a document when it is regenerated on disk. For example, macOS Preview may keep showing an older PDF until the window is focused, the file is reopened, or the application is restarted.

For smoother live preview, use a PDF viewer that reloads the file when it changes. On macOS, Skim is a good option. Other platforms have similar auto-reloading viewers, which are useful when working with tools that repeatedly rebuild PDFs.

Serve

Use `calepin serve` to preview a compiled website directory locally. This is useful for checking routing, assets, Pagefind search, and links after a static build.

```
calepin serve docs
calepin serve public
```

By default, Calepin uses `127.0.0.1` and the first available port from 8000. Set the host and port explicitly when you need a stable preview URL:

```
calepin serve docs --host 127.0.0.1 --port 8001
```

Open the served website in your default browser:

```
calepin serve docs --open
```